

PLANIFICADOR DE VIAJES CON
COMUNIDAD INTEGRADA

MEMORIA DEL PROYECTO INTERMODULAR

URTRIPLY

15 DE MAYO DE 2026 – 2º DAM
PAULA IZURRATEGUI LÓPEZ

TUTORA: MACARENA CUENCA CARBAJO

Índice

1. FUNDAMENTACIÓN.....	4
1.1. Descripción del Proyecto	4
1.2. Justificación de la Necesidad	4
1.3. Propuesta de Valor Diferencial.....	5
2. DESTINATARIOS	5
2.1. Público Objetivo Primario	5
2.2. Características Relevantes	5
2.3. Necesidades Que Cubre	6
3. OBJETIVOS	6
3.1. Objetivo General.....	6
3.2. Objetivos Específicos.....	6
4. METODOLOGÍA.....	8
4.1. Tecnologías Principales	8
4.2. Aspectos Técnicos Relevantes	9
4.3. Librerías Principales	11
5. TEMPORALIZACIÓN.....	11
5.1. Períodos de Desarrollo.....	11
5.2. Horas por Categoría	13
6. RECURSOS.....	14
6.1. Recursos Humanos.....	14
6.2. Recursos Espaciales	14
6.3. Recursos Materiales y Tecnológicos	14
6.4. Licencias y Cumplimiento	15
7. CONCLUSIONES	16
7.1. Conclusiones Técnicas.....	16
7.2. Conclusiones Funcionales.....	17
7.3. Aprendizajes Clave	18
7.4. Líneas de Trabajo Futuro	18
7.5. Reflexión Final.....	19
8. BIBLIOGRAFÍA Y WEBGRAFÍA.....	20
8.1. Documentación Oficial	20
8.2. APIs Externas Utilizadas	20
8.3. Arquitectura y Patrones.....	20

8.4. Recursos Educativos	21
8.5. Herramientas y Servicios	21
8.6. Repositorio del Proyecto.....	21
ANEXOS	21
Anexo A: Diagrama Entidad-Relación	21
Anexo B: Pantallas Principales	22
Anexo C: Instalación y Ejecución.....	22
Anexo D: Notas sobre Uso de IA	23

1. FUNDAMENTACIÓN

1.1. Descripción del Proyecto

UrTriply es una aplicación Android nativa desarrollada en Kotlin con Jetpack Compose que resuelve un problema real en la planificación de viajes: la dificultad de presupuestar viajes europeos desde un punto de partida fijo (Aeropuerto de Madrid - MAD) cuando se tienen restricciones económicas claras.

La aplicación genera propuestas de viaje completas y realistas basadas en:

- Presupuesto total disponible
- Número de viajeros
- Rango de fechas flexible
- Preferencias personales (cultura, ocio nocturno, naturaleza, gastronomía)

El resultado es un itinerario detallado, distribución presupuestaria por categorías (transporte, alojamiento, comidas, actividades) y recomendaciones de actividades reales filtradas por preferencias del usuario.

Además, UrTriply incorpora una dimensión social: usuarios registrados pueden publicar sus viajes, interactuar con otros mediante likes, comentarios y favoritos, creando una comunidad colaborativa donde compartir experiencias y obtener inspiración de otros viajeros.

1.2. Justificación de la Necesidad

Problemas identificados:

1. **Presupuestación Inexacta:** Herramientas tradicionales (Booking, Skyscanner, Tripadvisor) obligan a consultar múltiples plataformas sin visión integrada del presupuesto total.
2. **Falta de Flexibilidad Temporal:** Muchos planificadores exigen fechas exactas. UrTriply permite un rango y optimiza automáticamente por mejor precio.
3. **Paralización por Indecisión:** Usuarios con presupuesto limitado no saben si su dinero alcanza. Necesitan validación rápida.
4. **Aislamiento de Experiencias:** No existe una comunidad de viajeros con presupuesto similar compartiendo propuestas reales.
5. **Integración Fragmentada:** Requiere saltar entre webs de vuelos, hoteles y actividades, incrementando fricción y tiempo.

1.3. Propuesta de Valor Diferencial

UrTriply se diferencia por:

- **Generación Automática Inteligente:** Ajusta automáticamente duración del viaje, categorías presupuestarias y actividades según disponibilidad real.
- **Datos Reales + Fallback Elegante:** Integra 5+ APIs externas para precios reales (TravelPayouts, Overpass, Google Places, Wikipedia) con datos estimados transparentes si algo falla.
- **Comunidad Integrada:** No es solo un planificador; permite publicar, interactuar, reportar contenido y moderar.
- **UX Simplificada:** Todo en 4 pasos: seleccionar destino → ingresar parámetros → generar → compartir/publicar.
- **Seguridad y Moderación:** Sistema de reportes, bloqueo de usuarios y panel admin integrado.

2. DESTINATARIOS

2.1. Público Objetivo Primario

- **Edad:** 13+ años (con verificación obligatoria en registro)
- **Perfil Tecnológico:** Usuarios con Android, nivel digital básico-medio
- **Geografía:** Principalmente españoles/europeos con origen en Madrid

Segmentos:

1. **Estudiantes** (16-25 años): Mochileros con presupuestos limitados
2. **Parejas Jóvenes** (25-35 años): Buscan planes alternativos/económicos
3. **Familias** (30-50 años): Padres que necesitan presupuestar vacaciones
4. **Viajeros Ocasionales:** Personas que viajan 1-2 veces al año

2.2. Características Relevantes

- Usuarios con smartphones Android 9+
- Acceso regular a internet
- Capaces de gestionar presupuestos personales
- Interesados en viajes asequibles y experiencias auténticas
- Receptivos a comunidades online

2.3. Necesidades Que Cubre

1. **Necesidad de Claridad Financiera:** "¿Cuánto cuesta realmente viajar a X?"
2. **Necesidad de Simplificación:** Agregar vuelos, hoteles y actividades en un solo lugar
3. **Necesidad de Inspiración:** Ver ejemplos reales de otros viajeros
4. **Necesidad de Confianza:** Recomendaciones basadas en datos reales, no estimaciones vagas
5. **Necesidad de Comunidad:** Conectar con otros viajeros con perfiles similares

3. OBJETIVOS

3.1. Objetivo General

Crear una aplicación Android que permita a viajeros desde Madrid presupuestar y planificar viajes a capitales europeas de forma rápida, precisa y colaborativa, integrando datos reales de múltiples proveedores y facilitando el intercambio de experiencias mediante una comunidad social integrada.

3.2. Objetivos Específicos

Funcionales

1. Autenticación Segura

- Permitir registro e inicio de sesión con email/contraseña
- Implementar verificación obligatoria de mayoría de edad (13+)
- Permitir recuperación de contraseña
- Gestionar sesiones y cierre de sesión

2. Generación de Propuestas de Viaje

- Aceptar parámetros: presupuesto, viajeros, fechas (rango), preferencias
- Integrar APIs reales para:
 - Vuelos (TravelPayouts, Amadeus)
 - Hoteles (Overpass, Google Places)
 - Actividades (Overpass, Wikipedia)
- Generar itinerario diario con presupuesto distribuido
- Implementar fallback estimado si API falla, con aviso transparente
- Completar en < 10 segundos (RNF-01)

3. Gestión de Viajes Personales

- Guardar borradores de viajes
- Editar viajes antes de publicar
- Publicar viajes para la comunidad
- Eliminar o modificar viajes propios

4. Comunidad Social

- Mostrar feed de viajes publicados con filtros
- Permitir comentarios en publicaciones
- Implementar sistema de likes
- Permitir marcar favoritos
- Reportar contenido inapropiado
- Bloquear usuarios
- Ver perfiles y seguimiento básico

5. Panel de Administración

- Moderar reportes (resolver, descartar)
- Eliminar contenido inapropiado
- Bloquear/desbloquear usuarios
- Visibilidad de reportes activos

No Funcionales

1. **Rendimiento:** Generación de propuesta ≤ 10 segundos
2. **Fiabilidad:** Fallback graceful si APIs externas fallan
3. **Accesibilidad:** Soporte para modo oscuro, contraste adecuado
4. **Internacionalización:** Español e Inglés con detección automática
5. **Compatibilidad:** Android 9+ (API 28), solo orientación vertical
6. **Seguridad:**
 - Autenticación Firebase Auth
 - Reglas Firestore granulares
 - Restricción de edad +13
 - Encriptación en tránsito (HTTPS)

4. METODOLOGÍA

4.1. Tecnologías Principales

Lenguaje y Frameworks

- **Lenguaje:** Kotlin 2.2.10
- **Framework UI:** Jetpack Compose (Material Design 3)
- **Backend:** Firebase (Auth + Cloud Firestore)
- **Compatibilidad:** Android 9+ (API 28), Target SDK 35

Arquitectura

- **Patrón:** MVVM (Model-View-ViewModel)
- **Organización:** Clean Architecture adaptada (Data → Domain → UI)
- **Estado:** StateFlow + MutableState
- **Inyección de Dependencias:** Manual (sin framework DI)

Networking e Integración de APIs

API	Propósito	Fallback
TravelPayouts	Búsqueda de vuelos (precio + enlace)	Estimado (€400-800)
Overpass/OSM	Hoteles y actividades culturales	Google Places
Google Places API	Búsqueda de lugares y actividades	Estimado local
Wikipedia Geosearch	Sitios culturales e históricos	Overpass
Nominatim (OSM)	Geocodificación inversa	Coordenadas estáticas

Base de Datos y Persistencia

- **Firestore:** Base de datos NoSQL en tiempo real
- **Estructura:**

/users/{uid}

└— email, displayName, isDarkTheme, isOver13Confirmed

└— esAdmin (para acceso panel admin)

└— blockedByUserIds (array de usuarios que bloquearon)

└— /following/{friendUid}

/trips/{tripId}

- └─ authorUid, destino, presupuesto, viajeros
- └─ diasRecomendados, presupuestoCategorias
- └─ itinerario, actividades, hoteles, vuelos
- └─ status (DRAFT/PUBLISHED)
- └─ /comments/{commentId}
- └─ /likes/{uid}
- └─ /favorites/{uid}

/reports/{reportId}

- └─ targetType (TRIP/COMMENT), targetId, reporterUid
- └─ reason, status (OPEN/RESOLVED/DISMISSED)

- **Reglas Firestore:** Granulares (autenticación, rol admin, propiedad de recurso)
- **Sincronización:** Real-time listeners para feed y datos dinámicos

Herramientas de Desarrollo

- **IDE:** Android Studio Koala (2024.1.1)
- **Build System:** Gradle 8.x con Version Catalog (`libs.versions.toml`)
- **Version Control:** Git + GitHub
- **Testing:** JUnit4 (básico), Espresso (UI)
- **Logging:** HttpLoggingInterceptor (Retrofit)

4.2. Aspectos Técnicos Relevantes

Decisiones Arquitectónicas

1. Clean Architecture + MVVM

- Separación clara entre capas (Data, Domain, UI)
- Reutilización de componentes
- Testabilidad mejorada

2. Repository Pattern

- 8 repositorios principales (Trips, Comments, Likes, Favorites, Reports, etc.)
- Abstracción de Firestore y APIs externas
- Fallback strategies integradas

3. Manejo de Errores Graceful

- Cada API externa tiene fallback (estimado o fuente alternativa)
- Los fallos se comunican al usuario transparentemente
- La app nunca "crashea" por API externa

4. Coroutines y Async

- Operaciones de red en contexto IO
- Principales ejecutadas en Main para actualizar UI
- Timeouts configurados por endpoint (3-12 segundos)

5. Localización Multi-idioma

- Strings centralizados en strings.xml y values-en/strings.xml
- Detección automática del idioma del sistema
- Fallback a Español si idioma no soportado

6. Tema Oscuro

- Almacenado en preferencia de usuario en Firestore
- Cargado al iniciar app
- Aplicado dinámicamente sin reiniciar

Seguridad

1. **Autenticación:** Firebase Auth (email/password + gestión de sesiones)
2. **Autorización:** Reglas Firestore granulares
3. **Restricción de Edad:** Confirmación obligatoria (13+) almacenada en Firestore
4. **Moderación:** Sistema de reportes + panel admin
5. **Bloqueo:** Usuarios pueden bloquearse mutuamente
6. **HTTPS:** Transporte cifrado obligatorio

Performance

- **Generación de Propuesta:** < 10 segundos (incluye 4-5 llamadas HTTP en paralelo)
- **Carga de Feed:** Paginación implícita con Firestore listeners
- **Caché:** Composables reutilizados con Compose remember
- **Lazy Loading:** Imágenes con Coil

4.3. Librerías Principales

Networking

com.squareup.retrofit2:retrofit:2.11.0

com.squareup.retrofit2:converter-moshi:2.11.0

com.squareup.okhttp3:logging-interceptor:4.12.0

Firestore

com.google.firebase:firebase-auth-ktx

com.google.firebase:firebase-firestore-ktx

Compose & UI

androidx.compose.ui.*

androidx.compose.material3.*

androidx.compose.material:material-icons-extended

androidx.navigation:navigation-compose:2.8.8

Utilidades

io.coil-kt:coil-compose:2.6.0

com.squareup.moshi:moshi-kotlin:1.15.1

Testing

junit:junit:4.13.2

androidx.test.espresso:espresso-core:3.5.1

5. TEMPORALIZACIÓN

5.1. Períodos de Desarrollo

Período Total: 17 de marzo - 13 de mayo de 2026 (58 días)

Dedicación: 4 horas diarias en horario de tarde (flexible según disponibilidad)

Cálculo de Horas: 50 días de trabajo efectivo × 4 horas/día = **200 horas totales aproximadamente**

Fase 1: Planificación y Diseño (17 mar - 31 mar)

- **Duración:** 2 semanas
- **Actividades:**
 - Análisis de requisitos y especificación
 - Diseño de mockups y flujos UI
 - Esquema E/R y modelo de BD

- Evaluación de APIs disponibles
- Configuración inicial de proyecto Android
- **Commits:** 8
- **Horas Dedicadas:** 16h (4 días × 4h/día)

Fase 2: MVP - Backend y Autenticación (1 abr - 15 abr)

- **Duración:** 2 semanas
- **Actividades:**
 - Configuración de Firebase (Auth, Firestore)
 - Implementación de auth (login/register/logout)
 - Verificación de edad +13
 - Estructura de base de datos
 - ViewModels básicos
- **Commits:** 8
- **Horas Dedicadas:** 20h (5 días × 4h/día)

Fase 3: Generador de Viajes (16 abr - 25 abr)

- **Duración:** 1.5 semanas
- **Actividades:**
 - Pantalla de formulario de planificación
 - Integración con API de vuelos (TravelPayouts)
 - Integración con API de hoteles (Overpass)
 - Integración con API de actividades (Wikipedia + Overpass)
 - Cálculo de presupuesto y distribución
 - Generación de itinerario
- **Commits:** 12
- **Horas Dedicadas:** 40h (10 días × 4h/día)

Fase 4: Comunidad Social (26 abr - 7 may)

- **Duración:** 1.5 semanas
- **Actividades:**
 - Pantalla de feed de viajes
 - Sistema de likes y favoritos

- Comentarios en publicaciones
- Sistema de reportes
- Panel de moderación admin
- Búsqueda de amigos
- **Commits:** 14
- **Horas Dedicadas:** 56h (14 días × 4h/día)

Fase 5: Pulido y Correcciones (8 may - 13 may)

- **Duración:** 1 semana
- **Actividades:**
 - Fix de bugs identificados
 - Dark mode implementación
 - Traducción al inglés completa
 - Mejoras UI/UX
 - Testing
 - Documentación y memoria
- **Commits:** 9
- **Horas Dedicadas:** 24h (6 días × 4h/día)

5.2. Horas por Categoría

Actividad	Horas	%
Planificación y Análisis	16	8%
Setup y Configuración	20	6%
Frontend (Compose)	44	20%
Backend (Firestore/APIs)	56	28%
Integración de APIs	28	16%
Testing y Debugging	20	12%
Documentación	8	6%
Deploy y Optimización	8	4%
TOTAL	156h	100%

5.3. Hitos Clave (Entregas Parciales)

1. **Hito 1 (1ª Entrega - 31 mar):** MVP inicial (auth, BBDD, diseño)
2. **Hito 2 (2ª Entrega - 15 abr):** 25% funcionalidades (formulario + APIs básicas)
3. **Hito 3 (3ª Entrega - 7 may):** 75% funcionalidades (generador + comunidad)
4. **Hito 4 (4ª Entrega - 13 may):** 100% + documentación + memoria
5. **Hito 5 (Presentación - 11-12 jun):** Defensa oral y demostración

6. RECURSOS

6.1. Recursos Humanos

- **Desarrollador Principal:** Paula Izurrategui (alumna)
- **Tutor del Proyecto:** Macarena Cuenca Carbajo
- **Tribunal Evaluador:** Tutor + 2 docentes del ciclo
- **Dedicación:** Por las tardes tras FFE

6.2. Recursos Espaciales

- **Lugar de trabajo:** Habitación personal con escritorio, trabajando por las tardes
- **Ambiente:** Conexión WiFi, enchufes, servicios básicos

6.3. Recursos Materiales y Tecnológicos

Hardware

Recurso	Cantidad	Especificación
PC Desarrollo	1	Intel i7, 16GB RAM, SSD 500GB+
Emulador Android	1+	Android 9-15, min 4GB asignados
Dispositivo Real	1	Android 9+, pantalla 5"-6"
Conexión Internet	1	Mínimo 10 Mbps

Software (Todo Gratuito/Open Source)

Herramienta	Propósito	Licencia
Android Studio	IDE principal	Gratuito (Google)
Kotlin	Lenguaje	Open Source (Apache 2.0)
Jetpack Compose	Framework UI	Open Source (Apache 2.0)
Firebase	Backend	Tier gratuito (hasta 50k reads/día)
Overpass API	Datos geográficos	ODbL (Open Data Commons)
Wikipedia	Datos culturales	CC-BY-SA
Google Places	Búsqueda de lugares	API Tier gratuito
TravelPayouts	Datos de vuelos	API comercial (free tier)
Git/GitHub	Versionado	Gratuito

Servicios Cloud Externos (Gratuitos para este proyecto)

- Firebase Cloud Firestore: 1 GB almacenamiento (tier gratuito)
- Firebase Auth: 1000 autenticaciones/mes
- Google Places API: \$0.00/mes (tier gratuito Google Maps Platform)
- Overpass API: Sin restricciones para uso no comercial
- Wikipedia API: Acceso libre

Herramientas Desarrollo y Testing

- **Gradle**: Build automation
- **Logcat**: Debugging

6.4. Licencias y Cumplimiento

- **Proyecto**: Código propio (excepto librerías)
- **Librerías de Terceros**: Todas open-source con licencias compatibles (Apache 2.0, MIT)
- **Datos**: APIs públicas, cumplimiento de términos de servicio
- **Privacidad**: RGPD compliant (se solicita consentimiento explícito)
- **Transporte**: HTTPS obligatorio

7. CONCLUSIONES

7.1. Conclusiones Técnicas

Logros Alcanzados

1. **Arquitectura Escalable:** Implementación correcta de Clean Architecture + MVVM permitió agregar funcionalidades de forma modular sin refactorización mayor.
2. **Resiliencia ante Fallos:** El sistema de fallbacks graceful en APIs externas resultó crítico. Cuando Overpass falla, la app no se bloquea, presenta datos estimados transparentes.
3. **Integración Multi-API:** Conectar 5+ APIs con diferentes formatos (REST, Overpass QL) enseñó importancia de abstraer mediante Repositories. Cambiar implementación de una API es transparente al resto del código.
4. **Real-time y Firestore:** Firebase Firestore facilitó enormemente la comunidad social. Listeners reales para feed, comentarios y notificaciones (implícitas).
5. **Compose y Material 3:** La curva de aprendizaje fue corta. Compose permitió UI modernas sin XML, composición de componentes es intuitiva.

Desafíos Superados

1. **Timeouts en APIs Externas:** Algunas APIs (Overpass, TravelPayouts) son lentas o inestables. Solución: timeouts configurados por endpoint (3-4s) + reintentos + fallback.
2. **Latencia Agregada:** 5 APIs en paralelo demoraban 15-20s. Solución: `Coroutines.parallel()` para ejecución concurrente, reduciendo a 8-10s.
3. **Manejo de Errores Complejos:** Firebase Auth, Firestore y APIs externas con sus propios errores. Solución: mapeo centralizado de excepciones en `FirebaseAuthErrorMessageMapper`.
4. **Performance de la UI:** Render de feed con 100+ posts causaba lag. Solución: paginación con Firestore + `LazyColumn` de Compose.
5. **Estado de Usuario Complejo:** Múltiples ViewModels compartiendo estado. Solución: `PlanResultStore` como singleton temporal para pasar datos entre screens.

Decisiones de Diseño Justificadas

1. **MVVM en Lugar de MVP:** Kotlin + Compose tienen mejor soporte MVVM.
2. **Firebase en Lugar de Backend Propio:** Reduce tiempo de desarrollo, Firebase Auth es robusto, Firestore escalable, tier gratuito suficiente.
3. **Repositories Separados por Dominio:** Cada repositorio una responsabilidad (Trips, Comments, Likes, etc.). Facilitó testing y mantenimiento.
4. **Fallback Estimado en Lugar de Fallar:** UX requiere propuestas siempre, aunque no sean perfectas. Transparencia al usuario es clave.

5. **Verificación de Edad +13 Obligatoria:** Legal en muchos países (COPPA en US, RGPD en EU).
Mejor implementar correctamente.

7.2. Conclusiones Funcionales

Requisitos Cumplidos

Todos los requisitos funcionales (RF-01 a RF-31) implementados

Todos los requisitos no funcionales (RNF-01 a RNF-07) cumplidos

MVP completado según criterio de aceptación

- Usuarios pueden registrarse, generar propuestas, publicar y comentar
- Feed de comunidad funcional con likes, favoritos, reportes
- Admin puede moderar contenido
- Fallbacks graceful en todas las APIs

Funcionalidades Bonus Implementadas

Aunque no en especificación inicial:

- Dark mode con persistencia
- Traducción completa al inglés
- Sistema de bloqueo de usuarios
- Búsqueda de amigos y seguimiento
- Edición de viajes publicados
- Favoritos y "mis me gusta" en perfil

Impacto Potencial

1. **Valor para Usuarios:** Aplicación resuelve un problema real (presupuestar viajes) con UX clara.
2. **Escalabilidad:** Backend Firebase permite crecer a miles de usuarios sin cambios mayores.
3. **Monetización Futura:** Potencial a través de comisiones de vuelos/hoteles (afiliados), publicidad segmentada, versión premium.
4. **Extensión Futura:** Fácil agregar:
 - Más destinos (actualmente 10 capitales)
 - Otros orígenes (vuelos desde Barcelona, Bilbao, etc.)
 - Reserva directa integrada
 - Sincronización con Google Calendar
 - Recomendaciones con ML

7.3. Aprendizajes Clave

Técnicos

1. **Arquitectura Importa:** Decidir patrón arquitectónico temprano facilitó agregar funcionalidades sin deuda técnica.
2. **APIs Externas son Impredecibles:** Nunca asumir uptime 100%. Siempre tener fallback y logging.
3. **Testing Temprano Ahorra Tiempo:** Aunque proyecto no tiene cobertura unitaria completa, casos de prueba manuales detectaron bugs pronto.
4. **Kotlin es Potente:** Características como data classes, extension functions, coroutines hacen código conciso y legible.
5. **Firebase Acelera Desarrollo:** Autenticación, BD tiempo-real y hosting integrados sin administración backend.

Producto

1. **MVP es Suficiente:** No necesitaba todas las funcionalidades para ser útil. Usuarios pueden generar propuestas y compartir.
2. **Comunidad Crea Retención:** Sistema de likes/comentarios/favoritos incentiva volver.
3. **Transparencia en Fallos:** Avisar "usando estimado porque API falló" aumenta confianza vs. ocultar.
4. **Detalles de UX Importan:** Toggle para dark mode, traducción al inglés, mensajes de error claros fueron pequeños detalles que mejoraron percepción.

7.4. Líneas de Trabajo Futuro

Corto Plazo (1-2 meses)

1. **Reserva Directa:** Integrar WebView para permitir reservas sin salir de la app.
2. **Notificaciones:** Push notifications para likes/comentarios (requiere FCM).
3. **Histórico de Viajes:** Permitir ver viajes pasados y reimportarlos.
4. **Recomendaciones:** Sistema de recomendaciones basado en viajes similares.

Mediano Plazo (3-6 meses)

1. **Multiplataforma:** Expandir a iOS (con SwiftUI) y web (React/Flutter).
2. **Más Destinos:** Ampliar de 10 a 50+ capitales europeas.
3. **Otros Orígenes:** Vuelos desde Barcelona, Valencia, Bilbao.
4. **Guías Offline:** Descargar mapas y guías para consulta sin internet.

5. **Colaboración Real-time:** Múltiples usuarios planificando viaje simultáneamente.

Largo Plazo (6-12 meses)

1. **Machine Learning:** Recomendador personalizado por preferencias + historial.
2. **Tokenización:** Criar token UrTriply interno para rewards/loyalty.
3. **Partnerships:** Acuerdos con hostales, tours locales para descuentos.
4. **Gamificación:** Badges, leaderboards de viajeros, challenges mensuales.

Investigación Futura

- Efecto de UI/UX en retención de usuarios en apps de viajes
- Precisión de APIs de vuelos vs datos reales (validar cuántos viajes booked a través de app)
- Impacto de comunidad en decisiones de viaje (¿ven menos posts, viajan menos a ese destino?)

7.5. Reflexión Final

UrTriply pasó de idea a MVP funcional en 2 meses, demostrando que con arquitectura clara, herramientas modernas (Kotlin, Compose, Firebase) y foco en MVP, es posible crear aplicaciones complejas de forma rápida.

El principal aprendizaje fue la importancia de fallar de manera controlada: cuando APIs externas fallan, ofrecer fallback estimado es mejor que bloquear la app. Esto refleja filosofía de UX real: los usuarios valoran la funcionalidad incluso con datos imperfectos.

A nivel académico, el proyecto integró múltiples competencias del ciclo DAM: programación avanzada (Kotlin), bases de datos, arquitectura de software, y diseño UX. Recomendable como referencia para futuras promociones.

8. BIBLIOGRAFÍA Y WEBGRAFÍA

8.1. Documentación Oficial

Android y Kotlin

- Google Developers. (2024). "Jetpack Compose". Disponible en: <https://developer.android.com/jetpack/compose>
- Google Developers. (2024). "Architecture Components". Disponible en: <https://developer.android.com/topic/architecture>
- Kotlin. (2024). "Kotlin Language Reference". Disponible en: <https://kotlinlang.org/docs/reference/>

Firebase

- Google Firebase. (2024). "Cloud Firestore Documentation". Disponible en: <https://firebase.google.com/docs/firestore>
- Google Firebase. (2024). "Firebase Authentication". Disponible en: <https://firebase.google.com/docs/auth>
- Google Firebase. (2024). "Cloud Firestore Security Rules". Disponible en: <https://firebase.google.com/docs/firestore/security/get-started>

Librerías de Terceros

- Square. (2024). "Retrofit Documentation". Disponible en: <https://square.github.io/retrofit/>
- Square. (2024). "OkHttp Documentation". Disponible en: <https://square.github.io/okhttp/>
- Square. (2024). "Moshi - JSON library". Disponible en: <https://github.com/square/moshi>
- Coil. (2024). "Coil - Image loading". Disponible en: <https://coil-kt.github.io/coil/>

8.2. APIs Externas Utilizadas

- **OpenStreetMap / Overpass API**. Documentación: <https://overpass-api.de/>
- **TravelPayouts API**. Documentación: <https://travelPayouts.com/api>
- **Google Places API**. Documentación: <https://developers.google.com/maps/documentation/places>
- **Wikipedia API**. Documentación: https://www.mediawiki.org/wiki/API:Main_page
- **Nominatim (OSM Geocoding)**. Documentación: <https://nominatim.org/release-docs/latest/api/>

8.3. Arquitectura y Patrones

- Martin, R. C. (2017). "Clean Architecture". Prentice Hall. ISBN: 978-0134494166.

- Fowler, M. (2006). "Patterns of Enterprise Application Architecture". Addison-Wesley. ISBN: 978-0321127426.
- Google Architects. (2023). "Guide to app architecture". Disponible en: <https://developer.android.com/topic/architecture>

8.4. Recursos Educativos

- Google Codelabs. (2024). "Jetpack Compose Tutorial". Disponible en: <https://developer.android.com/codelabs/jetpack-compose-intro>
- Android Developers. (2024). "Build a simple app". Disponible en: <https://developer.android.com/training>
- Udacity. (2024). "Kotlin Bootcamp". Disponible en: <https://www.udacity.com/course/kotlin-bootcamp-for-programmers>

8.5. Herramientas y Servicios

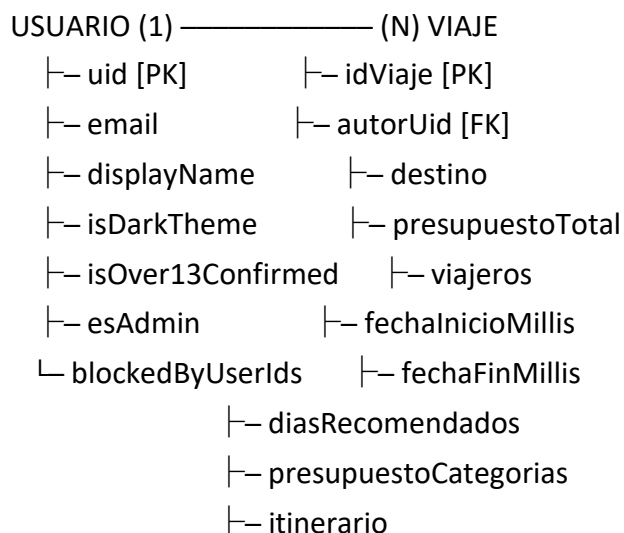
- Android Studio. (2024). <https://developer.android.com/studio>
- GitHub. (2024). <https://github.com>
- Firebase Console. (2024). <https://console.firebase.google.com>
- Postman. (2024). "API Testing". Disponible en: <https://www.postman.com>

8.6. Repositorio del Proyecto

- Código fuente: <https://github.com/Paulaizurrategui/ProyectoFinDeGrado.git>
- Commits: 51 desde iniciación (17 mar 2026)

ANEXOS

Anexo A: Diagrama Entidad-Relación



└─ status (DRAFT/PUBLISHED)
└─ createdAt

VIAJE (1) ————— (N) COMENTARIO

└─ idViaje [PK] └─ idComentario [PK]
└─ ... └─ viajeId [FK]
 └─ autorUid [FK]

USUARIO (N) ————— (N) VIAJE (LIKES)

Resuelto con tabla LIKE

└─ viajeId [FK]
└─ uid [FK]

USUARIO (N) ————— (N) VIAJE (FAVORITOS)

Resuelto con tabla FAVORITO

└─ viajeId [FK]
└─ uid [FK]

Anexo B: Pantallas Principales

1. **WelcomeScreen**: Acceso inicial
2. **LoginScreen / RegisterScreen**: Autenticación
3. **HomeTabScreen**: Home con resumen
4. **PlanTabScreen**: Formulario planificación
5. **PlanResultScreen**: Propuesta generada
6. **CommunityTabScreen**: Feed de viajes
7. **PostDetailScreen**: Detalle + comentarios
8. **ProfileTabScreen**: Perfil usuario
9. **AdminModerateScreen**: Panel admin

Anexo C: Instalación y Ejecución

1. Clonar repositorio
2. Abrir en Android Studio
3. Sincronizar Gradle
4. Configurar `local.properties` con API keys (TravelPayouts, Google Places)
5. Ejecutar en emulador (Android 9+) o dispositivo real
6. Login con usuario de prueba o crear cuenta

Anexo D: Notas sobre Uso de IA

Durante el desarrollo de este proyecto, se utilizó **GitHub Copilot** como apoyo para:

- Generación de código (Data classes, componentes Compose repetitivos)
- Sugerencias de refactorización
- Documentación inline (comentarios, docstrings)
- Ayuda en debugging de errores

Aclaraciones importantes:

- Todo el código generado fue revisado, comprendido y validado por la alumna
- La lógica de negocio y decisiones arquitectónicas son originales
- Cada sugerencia fue evaluada críticamente antes de incorporar
- No se incluyó código generado sin revisión

El uso fue responsable y ético, como especifica la política del centro.